

MLFF PERF-P0 Native TARGET-DATA2B Specification

Exact native persistence, shared statistics, and matched qualification

mdstats project

2026-08-15

Contents

Status	1
Authority boundary	2
Scientific invariants	2
Correlation-unit-balanced weights	2
Weighted quantile convention	2
Scaled RMS distance	3
Fixed-reference-mass local radius	3
Native persistence contract	3
Schemas	3
Canonical arrays	4
Content-addressed layout	4
Memory mapping	5
Historical v1 compatibility and migration	5
Exact execution accelerations	5
Shared weight profiles	5
One ordering per column	5
Uniform-weight radius dispatch	5
Bulk family extraction	5
Dense exact qualification backend	5
Campaign integration	5
Qualification	6
Unit and regression evidence	6
Complete supplied-data CPU evidence	6
Acceptance decision	7
Limits and next gate	7
References	7

Status

Gate: PERF-P0

Release: mdstats 0.20.179a0

Implementation status: complete for bounded supplied-data TARGET-DATA2B authority

Next gate: PERF-P1

This specification freezes the implementation contract for exact, scalable TARGET-DATA2B construction and persistence. It changes storage and execution, not coverage mathematics, source membership, target/replay roles, or scientific policy.

The authoritative implementation is distributed across:

- `mdstats.training_data.target_coverage`;
- `mdstats.training_data.target_coverage_store`;
- `mdstats.training_data.campaign_cli`;
- `benchmarks/benchmark_mlff_perf_p0.py`; and
- `tests/test_mlff_perf_p0.py`.

Authority boundary

PERF-P0 contains two classes of change.

Area	Authority class	Allowed change
TARGET-DATA2B persistence	S	Versioned representation and exact migration
Weight/statistics reuse	E	Execution-only cache and sort reuse
Uniform-weight radius path	E	Exact order-statistic dispatch
Family extraction	E	Columnar/bulk construction
Neighbor qualification hook	E	Bounded exact comparison only

The following remain frozen:

- family membership and ordering;
- correlation-unit balancing;
- robust-scale and extent conventions;
- scaled RMS distance;
- the fixed reference-mass fraction $\beta = 1/128$;
- leave-one-out semantics;
- duplicate-point and tie behavior;
- FP64 numerical authority; and
- all downstream qualification decisions.

Approximate nearest-neighbor search, frame subsampling, changed tolerances, and changed scientific digests are outside this gate.

Scientific invariants

Correlation-unit-balanced weights

Let a family contain frames indexed by $i \in \{1, \dots, N\}$ and let $c(i)$ identify the DATA5 correlation unit containing frame i . Let $\mathcal{C}$ be the set of represented correlation units and let n_c be the number of family frames in unit c . The authoritative frame weight is

$$w_i = \frac{1}{|\mathcal{C}| n_{c(i)}}.$$

The implementation normalizes the resulting FP64 vector so that

$$\sum_{i=1}^N w_i = 1.$$

A reusable weight profile is identified by the label domain, ordered frame membership, ordered correlation-unit identities, weighting policy, and leave-one-out policy. Cache location and process settings are not part of this identity.

Weighted quantile convention

For scalar values x_i with nonnegative weights w_i , let π be a stable ascending ordering of the values and define

$$W_j = \sum_{k=1}^j w_{\pi(k)}, \quad W = W_N.$$

For $q \in [0, 1]$, the project quantile is the left-continuous weighted empirical inverse

$$Q(q) = x_{\pi(j^*)}, \quad j^* = \min\{j : W_j \geq qW\}.$$

PERF-P0 performs one stable mergesort ordering per scalar column and derives all required quantiles from the same cumulative mass. This convention is project-specific and is stated explicitly because statistical packages expose multiple inequivalent sample-quantile definitions [4].

For feature column d , the robust scale is

$$s_d = \max(Q_d(0.75) - Q_d(0.25), s_{\min}),$$

with the existing fallback sequence:

1. $Q_d(0.99) - Q_d(0.01)$;
2. $\max(\text{std}(x_d), 1)$; and
3. the configured positive floor $s_{\min}$.

Extent channels retain the frozen $Q(0.01)$ and $Q(0.99)$ convention unless the policy explicitly supplies another existing extent level.

Scaled RMS distance

For a D -component family vector x_i and scale vector s , define

$$\tilde{x}_{i,d} = \frac{x_{i,d}}{s_d},$$

and

$$d(i, j) = \frac{1}{\sqrt{D}} \|\tilde{x}_i - \tilde{x}_j\|_2.$$

`scipy.spatial.cKDTree` remains the authoritative nearest-neighbor execution backend. The kd-tree family originates in multidimensional associative search [5], while the SciPy implementation uses a sliding-midpoint variant and exposes exact queries when `eps=0` [6].

Fixed-reference-mass local radius

For reference frame i , leave-one-out neighbor mass is

$$\mu_i(j) = \frac{w_j}{1 - w_i}, \quad j \neq i.$$

Let σ_i order non-self neighbors by nondecreasing distance. The local radius is

$$r_i = d(\sigma_i(k_i)),$$

where

$$k_i = \min \left\{ k : \sum_{m=1}^k \mu_i(\sigma_i(m)) \geq \beta - 10^{-15} \right\}, \quad \beta = \frac{1}{128}.$$

For a mathematically uniform profile, $w_i = 1/N$, each non-self increment is $1/(N - 1)$. The exact one-based neighbor rank is therefore obtained by the same FP64 cumulative-mass/search rule used by the weighted oracle. The implementation does not substitute an algebraically simplified ceiling when that could alter floating-point boundary behavior.

Duplicate points and equal distances are valid. Self exclusion is by neighbor index, not by zero distance.

Native persistence contract

Schemas

The gate introduces these public schemas:

Record	Schema
Family reference	<code>mdstats.target-coverage-family.v2</code>
Domain reference	<code>mdstats.target-coverage-domain.v2</code>
Top-level reference	<code>mdstats.target-coverage-reference.v2</code>
Native manifest	<code>mdstats.target-coverage-native-manifest.v2</code>
Campaign pointer	<code>mdstats.mlff-campaign-target-coverage-native-pointer.v2</code>
Shared weight profile	<code>mdstats.target-coverage-native-weight-profile.v1</code>
Migration report	<code>mdstats.target-coverage-migration-report.v1</code>

The scientific coverage version remains unchanged. The persistence version is:

`mdstats.target-data2b.native-persistence.2026-08.v2`.

Canonical arrays

Every authoritative family array is:

- a NumPy array rather than a nested Python numeric structure;
- C-contiguous;
- little-endian;
- explicitly typed as `<i8` or `<f8`;
- read-only after construction; and
- identified by name, dtype, shape, byte count, finite-value metadata, native-byte SHA-256, and a content digest.

NPY is used because its format stores dtype, shape, order, and native binary array data and is designed to support memory mapping [1]. NumPy memory-mapped loading keeps an array on disk while exposing normal array slicing semantics [2]. Object arrays and pickle are forbidden.

Content-addressed layout

The store writes one directory per reference content digest:

```
records/
  target-coverage-<reference-digest>/
    manifest.json
    weight-<id>-frame-indices.npy
    weight-<id>-weights.npy
    domain-<id>-family-<id>-values.npy
    domain-<id>-family-<id>-scales.npy
    domain-<id>-family-<id>-local-radii.npy
```

Shared (`frame_indices`, `weights`) pairs are factored into one weight profile. Family manifests refer to the profile by content identity.

Each NPY file is written through a streaming SHA-256 writer, flushed, and `fsync`-completed before publication. SHA-256 follows the Secure Hash Standard [3]. The digest is an integrity identity; it is not a digital signature or a substitute for a trusted distribution channel.

The store writes into a temporary sibling directory and publishes the completed directory with same-filesystem replacement. A restore fails closed on:

- pointer, manifest, or file checksum mismatch;
- path traversal or a path outside the campaign root;
- missing files or byte-count mismatch;
- dtype, shape, byte-order, or native-byte identity mismatch;
- unknown or duplicate weight-profile identity;

- family, domain, or reference digest mismatch; or
- unsupported schema/persistence version.

Memory mapping

Arrays at or above the caller-supplied threshold restore with read-only `mmap_mode="r"`. The threshold and absolute path are execution settings. They do not enter the scientific digest.

Historical v1 compatibility and migration

Historical `mdstats.target-coverage-*.v1` nested-JSON records remain readable. Their supplied historical digest is verified before conversion.

A v1 record is migrated only by:

1. restoring the complete v1 reference;
2. constructing a clean v2 reference from the restored scientific fields;
3. comparing every scalar field, domain, stratum, family field, and numerical array with exact equality;
4. writing an authenticated migration report; and
5. publishing v2 only when `exact_match=true`.

The migration oracle uses `numpy.array_equal` for numerical arrays. It does not round, recalculate, or scalarize the arrays. A difference path fails the campaign operation.

Exact execution accelerations

Shared weight profiles

`_TargetCoverageBuildCache` reuses frame indices and balanced weights when the full profile identity agrees. Cached and uncached family content digests must be identical.

One ordering per column

`_weighted_column_statistics` evaluates the scale and optional extent quantiles from one stable ordering per feature column. It does not allocate an all-column order matrix.

Uniform-weight radius dispatch

`_uniform_reference_rank` returns a rank only when all FP64 weights are exactly equal. Otherwise the existing weighted cumulative-mass implementation is used. The old implementation remains the qualification oracle.

Bulk family extraction

The profile-backed path resolves the frame descriptor table once and builds columnar `values[N,D]` and `missing[N,D]` arrays. Generic providers retain the existing adapter path. PERF-P0 does not claim a new pair minimum-image reuse optimization; that reuse already existed upstream.

Dense exact qualification backend

`_local_reference_radii_dense_exact` is a bounded test/benchmark oracle. It computes exact blockwise pair distances and stable weighted accumulation. It is not selected as production authority by this gate. A backend change requires a separate deterministic qualification, especially for duplicate points and ties.

Campaign integration

`CampaignStore` writes `TargetCoverageReference` through the native store and keeps only the authenticated pointer in campaign JSON state. Existing inline v1 records remain readable. On first authoritative use, a v1 record is migrated atomically with its exact migration report.

The campaign contract signature binds the persistence version. Worker count, radius block size, cache enablement, and `mmap` threshold remain execution-only.

Qualification

Unit and regression evidence

The gate tests prove:

- canonical native array properties;
- exact v1-to-v2 migration;
- weight-profile deduplication;
- mmap restore;
- fail-closed array tamper detection;
- uniform-rank equality to the weighted oracle with duplicate/tied points;
- cached/uncached identity;
- one-sort quantile equality to the repeated-sort oracle;
- bounded dense/cKDTree numerical agreement; and
- worker/block-size exclusion from scientific identity.

Complete supplied-data CPU evidence

The matched benchmark uses all 27 supplied target XML sources:

- **37,633 frames**;
- **6,322,344 atoms**; and
- **263,398 family elements** across eight exact families.

All 48 persisted numerical-array identities match PERF-BASE0 exactly. Five isolated runs of each construction path produced one common scientific digest:

2f04aba96b876a10e62b7b0c22f26b544836005d11bc893177d4b29cbe4a3f82.

Path		Median wall	Observed range	Median process CPU	Median peak RSS
Pre-P0	exact	7.541 s	6.826–9.926 s	27.091 s	329.47 MiB
path					
PERF-P0	exact	6.236 s	5.818–8.253 s	26.043 s	328.50 MiB
path					

The matched median wall reduction is **17.30%**. Median process CPU decreases by **3.87%**. Construction peak RSS decreases by 0.97 MiB; the larger memory improvement occurs at persistence boundaries.

Representa- tion	Write wall	Read wall	Size	Write	RSS	incre- ment	Read	RSS	incre- ment
Nested JSON v1	10.366 s	14.382 s	42,749,676 B		167.77 MiB			189.35 MiB	
Native-array v2	0.184 s	0.180 s	17,912,666 B		0.12 MiB			28.02 MiB	

The observed native representation is **56.22x** faster to write, **79.70x** faster to read, and **58.10%** smaller. The exact v1/v2 migration report has no difference paths.

Machine-readable authority:

- audits/analysis/mlff_perf_p0_lta_cloud_cpu_2026-08-15.json;
- audits/analysis/mlff_perf_p0_lta_cloud_cpu_2026-08-15.md; and
- release/MLFF_PERF_P0_QUALIFICATION_0.20.179a0.json.

The benchmark separates XML ingestion from matched family construction so both paths receive the same in-memory arrays. Scheduling and page-cache state are observed rather than forcibly controlled; ranges are retained because timing noise is material.

Acceptance decision

PERF-P0 passes for the supplied-data scope because:

1. native v2 round-trips exactly and v1 remains readable;
2. v1/v2 migration is elementwise exact;
3. the uniform-weight path equals the old weighted oracle;
4. cached/uncached and worker/block-size variants preserve authority;
5. all PERF-BASE0 family-array hashes match;
6. matched construction time improves; and
7. native persistence materially reduces wall time, serialized size, and transient memory.

Limits and next gate

No MACE-MH-1 checkpoint, authorizing GPU runtime, or complete production campaign bundle was supplied. This gate therefore does not claim production DATA6 model-derived families, complete TARGET-DATA2C/DATA7 selection authority, DATA8 materialization, TRAIN2/EVAL2 timing, GPU memory, or OOM evidence.

PERF-P1 is next. It must consolidate exact deterministic FPS and progressive coverage state without changing the PERF-BASE0/P0 scientific oracle.

References

- [1] R. Kern, “A Simple File Format for NumPy Arrays,” NumPy Enhancement Proposal 1, 2007. Available at: <https://numpy.org/doc/1.13/neps/np-format.html> (accessed 2026-08-15).
- [2] NumPy developers, “numpy.load,” NumPy reference documentation. Available at: <https://numpy.org/doc/stable/reference/generated/numpy.load.html> (accessed 2026-08-15).
- [3] National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015. DOI: 10.6028/NIST.FIPS.180-4.
- [4] R. J. Hyndman and Y. Fan, “Sample Quantiles in Statistical Packages,” *The American Statistician* **50**(4), 361–365 (1996). DOI: 10.1080/00031305.1996.10473566.
- [5] J. L. Bentley, “Multidimensional Binary Search Trees Used for Associative Searching,” *Communications of the ACM* **18**(9), 509–517 (1975). DOI: 10.1145/361002.361007.
- [6] SciPy developers, “scipy.spatial.cKDTree,” SciPy reference documentation. Available at: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.html> (accessed 2026-08-15).